

Prueba Técnica – Desarrollador React Native

Contexto

Necesitamos evaluar tu capacidad para resolver problemas complejos en React Native con un enfoque profesional, escalable y de alto rendimiento. El siguiente ejercicio **parecerá sencillo a simple vista**, pero contiene múltiples capas de complejidad que pondrán a prueba tu dominio de la plataforma, manejo de estado, optimización, gestión nativa y arquitectura robusta.

Se valora especialmente el equilibrio entre calidad, tiempo de desarrollo y documentación de decisiones técnicas.

Enunciado – "Smart Feed"

Construye una aplicación móvil que muestre un **feed infinito de post multimedia** (imagen + texto) obtenido desde una API remota. El feed debe ser fluido (60 fps), funcionar offline progresivamente y permitir interacciones avanzadas sobre cada post.

Funcionalidades obligatorias

1. **Carga paginada e infinita**
2. Al llegar al final de la lista, se cargan más posts automáticamente.
 - Manejo de estados: `Loading`, `error`, `empty`, `retry`.
 - Control de concurrencia (evitar múltiples peticiones simultáneas).
3. **Pull-to-refresh**
4. Refresca el contenido y reinicia la paginación.
5. **Caché offline progresiva**
 - Los posts ya cargados deben persistir en disco y mostrarse instantáneamente al abrir la app sin conexión.
 - Mientras el usuario navega sin red, almacena las acciones (ej. "like") y sincronízalas automáticamente cuando vuelva la conectividad.
6. **Animaciones suaves**
 - Cada fila debe tener un efecto de *press feedback* y una animación de aparición (fade o slide).
 - El desplazamiento de la lista no debe entrecortarse aunque las imágenes sean grandes (optimización obligatoria).
7. **Soporte de orientación**
 - Rotación de dispositivo -> redimensionado inmediato de las imágenes sin recarga ni pérdida de posición del scroll.
8. **Accesibilidad**
 - Etiquetas `accessible`, `accessibilityLabel`, ajuste de contraste y soporte de VoiceOver/TalkBack.

Capas de complejidad

- **Gestión de memoria y rendimiento**
- Uso correcto de `FlatList`, `removeClippedSubviews`, `windowSize`, `getItemLayout`, `initialNumToRender`.
- Implementación de caché de imágenes con liberación de recursos (no `react-native-fast-image` por sí solo, sino una estrategia propia o justificada).
- **Manejo de estado avanzado**
- Debes elegir una solución (Redux Toolkit, Zustand, MobX, Recoil, Jotai, Context + `useReducer`, etc.) y justificarla.
- El estado offline + sincronización debe funcionar incluso con 10k posts almacenados.
- **Módulo nativo personalizado (opcional pero muy valorado)**
- Implementa un módulo nativo (iOS/Android) que exponga una función `getAverageColor(imageUri)` para extraer el color dominante de cada imagen y usarlo como fondo del texto del post.
- Si prefieres no hacerlo, explica cómo lo resolverías y muestra un mock funcional.
- **Pruebas**
- Al menos:
 - Test unitario de la lógica de paginación/offline.
 - Test de integración del componente principal (usando `@testing-library/react-native`).
- **Arquitectura**
- Debe ser modular, con separación clara de:
 - `API layer`, `Storage layer`, `UI components`, `Hooks (o conectores de estado)`, `Servicios (sync, network detection)`.
- **Extra: Gestos complejos**
- Al deslizar un post hacia la izquierda, debe aparecer una opción oculta (ej. "Compartir" o "Favorito") similar a un `Swipeable` altamente personalizado.

Requisitos técnicos explícitos

Área	Exigencia
React Native	Última versión estable (0.7x) o Expo SDK 5x.
Navegación	React Navigation 6.x (stack + tabs si aplica).
Persistencia	No usar AsyncStorage directamente. Elige entre: SQLite, WatermelonDB, Realm, MMKV, Redux Persist con esquema normalizado.
Manejo de imágenes	Proponer estrategia: precarga, almacenamiento en disco, cancelación de peticiones.
Detección de red	<code>NetInfo</code> + manejo de reconexión con backoff exponencial.
Optimización de renders	Evitar re-renderizados innecesarios en la lista. Cada post debe ser <code>React.memo</code> con comparación profunda controlada.

Animaciones

Usar `react-native-reanimated` (v2/v3) o `react-native-gesture-handler`.

API de ejemplo

Puedes usar cualquier API pública de posts con imágenes. Recomendamos **JSONPlaceholder** + fotos de Lorem Picsum, o construir un mock local con paginación simulada.

Obligatorio: la API debe tener latencia artificial de al menos 500ms para ver estados de carga, y un endpoint de fallo aleatorio (10% de errores) para probar reintentos.

Ejemplo de respuesta esperada:

```
{
  "page": 1,
  "posts": [
    {
      "id": "123",
      "title": "Post title",
      "imageUrl": "https://picsum.photos/id/1/600/400",
      "likes": 42,
      "createdAt": "2023-01-01T00:00:00Z"
    }
  ],
  "hasNextPage": true
}
```